

Algorithms and Data Structures

27.01.2025

Exercise 1 (4.0 points)

Concerning dynamic memory allocation in the C language, describe how you can define two-dimensional matrices of integer types. First, clarify how to store a 2D matrix into a 1D array and logically move from one data structure to the other. Then, describe how to perform dynamic memory allocation for 2D matrices.

Exercise 2 (1.5 points)

Given the following array of integer values, perform the first step of quicksort to sort the array in ascending order; then, from the initial array, generate the right and the left partitions.

11 12 13 5 8 0 2 3 1 7

Report three integer values: The pivot selected on the original array, the pivot you would select on the left partition generated from the original array, and the pivot you would select on the right partition generated (again) from the original array. No other symbols must be included in the response. The following is an example of the response format: 13 1 10

Exercise 3 (1.5 points)

Consider a binary tree whose visits return the following sequences.

Pre-order:	A	C	D	I	G	K	B	E	H	F	J
In-order:	I	D	C	K	G	A	E	H	B	F	J
Post-order:	I	D	K	G	C	H	E	J	F	B	A

Report the sequence of keys stored on the tree leaves, moving on the tree from left to right. Please report the list of keys on the same line, separated by a single space. No other symbols must be included in the response. The following is an example of the response format: A X C Y etc.

Exercise 4 (1.5 points)

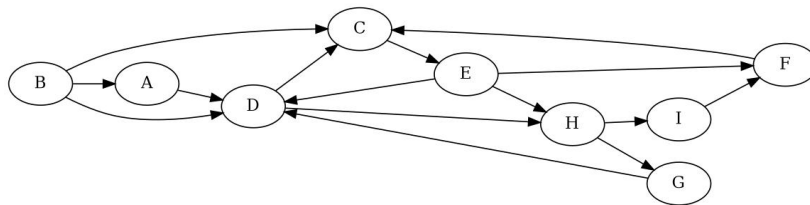
Insert the following sequence of keys into an initially empty hash table. The hash table has a size equal to $M=23$. Insertions occur character by character using open addressing with quadratic probing (with $c_1 = 1$ and $c_2 = 1$). Each character is identified by its index in the English alphabet (i.e., A=1, ..., Z=26). Equal letters are identified by a different subscript (i.e., A and A become A1 and A2).

G I G L I N G

Indicate in which elements the last three letters of the sequence are placed, i.e., I, N, and G, in this order. Please report your response as a sequence of integer values separated by one single space. No other symbols must be included in the response. The following is an example of the response format: 3 4 11

Exercise 5 (1.5 points)

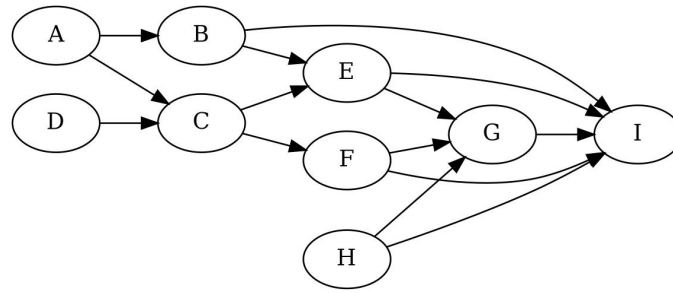
Visit the following graph depth-first, starting at node A. Label nodes with discovery and end-processing times. Start with the discovery time set to 1 on A. When necessary, consider nodes and edges in alphabetic order.



Display the end-processing time of all vertices. Please indicate the end-processing time of all vertices sorted in alphabetic order (i.e., display the end-processing time for A B C D etc.). Report a sequence of integer values separated by one single space. No other symbols must be included in the response. The following is an example of the response: 15 13 2 16 8 etc.

Exercise 6 (1.5 points)

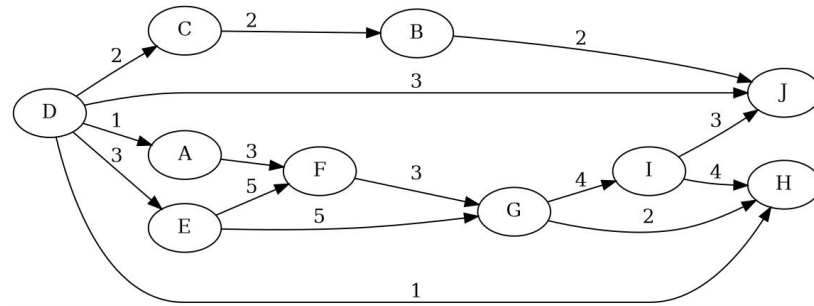
Given the following unweighted DAG, find the topological order of its vertices. Start the DFS visit from vertex A. If necessary, consider nodes in alphabetical order.



Report the direct topological order of the vertices. Report vertices on the same line separated by a single space. No other symbols must be included in the response. The following is an example of the response format: F A C H B etc.

Exercise 7 (1.5 points)

Given the following directed and weighted graph, apply Dijkstra's algorithm to find all shortest paths connecting node D with all the other nodes. When necessary, consider nodes and edges in alphabetical order.



Report the shortest paths to all vertices. Please indicate the shortest path to all vertices sorted in alphabetic order (i.e., display the shortest path for A B C D, etc.). Report a sequence of integer values separated by one single space. No other symbols must be included in the response. The following is an example of the response: 0 3 2 6 8, etc.

Exercise 8 (2.0 points)

Analyze the following program and indicate the exact output it generates. Please report the exact program output with no extra symbols.

```

void str(char *dest, char *str1, char *str2, char *str3) {
    while (*str1) {
        if (isupper(*str1)) {
            *dest = *str1;
            dest++;
        }
        str1++;
    }
    while (*str2) {
        if (isdigit(*str2)) {
            *dest = *str2;
            dest++;
        }
        str2++;
    }
    while (*str3) {
        if (islower(*str3)) {
            *dest = *str3;
            dest++;
        }
        str3++;
    }
    *dest = '\0';
}

int main() {
    char *str1 = "hello123WORLD";
    char *str2 = "aBcD123";
    char *str3 = "testCASE";
    char result[100];

```

```

    str(result, str1, str2, str3);
    printf("%s", result);
    return 0;
}

```

Exercise 9 (2.0 points)

The following code snippet illustrates a library manipulating points in the Cartesian space.

```

client.c
// LINE 1
int main (void) {
    point_t p1, p2;
    float d;
    p1 = new ();
    p2 = new ();
    set (p1, 0.0, 0.0, 0.0);
    set (p2, 10.0, 10.0, 10.0);
    d = dist (p1, p2);
    fprintf (stdout, "D = %f\n", d);
    disp (p1);
    disp (p2);
    return 1;
}

pointPublic.h
#ifndef _POINT_PUBLIC
#define _POINT_PUBLIC
#include <stdio.h>
typedef struct point_s *point_t;
point_t new ();
void set (point_t, float, float, float);
float dist (point_t, point_t);
void disp (point_t);
#endif

pointPrivate.h
#ifndef _POINT_PRIVATE
#define _POINT_PRIVATE
// LINE 2
#include <stdlib.h>
#include <math.h>
#include "pointPublic.h"
#endif

point.c
#include "pointPrivate.h"
point_t new () {
    ... allocate new point ...
}
void set (point_t p, float x, float y, float z) {
    ... set coordinates of a new point ...
}
float dist (point_t p1, point_t p2) {
    ... compute the distance between two points ...
}
void disp (point_t p) {
    ... free point p ...
}

```

Indicate which ones of the following statements are correct. Note that incorrect answers imply a penalty in the final score.

1. In Line 1 file `client.c` must include `pointPrivate.h`.
2. File `pointPublic.h` must include `pointPrivate.h`.
3. File `pointPrivate.h` must include `pointPublic.h`.
4. The structure `struct point_s` must be defined in `pointPublic.h`.
5. The structure `struct point_s` must be defined in `pointPrivate.h`.
6. The structure `struct point_s` must be defined in both `pointPublic.h` and `pointPrivate.h`.
7. If in Line 2 we include the library `stdio.h`, the pre-compiler will include it twice, and we will get multiple definitions and a linker error.

Exercise 10 (2.0 points)

Analyze the following program and indicate the exact output it generates. Please report the exact program output with no extra symbols.

```

void A(int, int);
void B(int, int);
void C(int, int);
void D(int, int);
int main() {

```

```

    A(3, 3);
    return 0;
}
void A(int counterA, int counterB) {
    if (counterA <= 0) return;
    printf("A");
    B(counterB, counterA - 1);
}
void B(int counterB, int counterC) {
    if (counterB <= 0) return;
    printf("B");
    C(counterC, counterB - 1);
    D(counterC - 1, counterB - 1);
}
void C(int counterC, int counterD) {
    if (counterC <= 0) return;
    printf("C");
    D(counterD, counterC - 1);
}
void D(int counterD, int counterA) {
    if (counterD <= 0) return;
    printf("D");
    A(counterA, counterD - 1);
}

```

Exercise 11 (3.0 points)

Write the following C function:

```
void lcp (char **string, int n);
```

To find the Longest Common Prefix among strings. The function receives an array of pointers to strings (`string`); `n` indicates the number of strings. Each string has a maximum length of 20 characters. The function finds the longest common prefix among the strings.

For example, if the input strings are the following {`''flower''`, `''flow''`, `''flight''`}, the function must display (on the standard output) `''fl''`.

Write the entire function using standard C libraries but implement all required personal libraries. Modularize the function adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward code, complex or impossible to understand, will be penalized in terms of the final evaluation.

Exercise 12 (5.0 points)

Write the following C function:

```
listB_t *merge (listA_t *head);
```

That receives a list of lists and returns a simple linked list. The main list (pointed by `head`) contains elements of type `listA_t`. Each `listA_t` element contains a pointer to a secondary list of elements of type `listB_t`. Each `listB_t` element stores a string and a pointer to another `listB_t` element. All secondary lists are sorted alphabetically. The function must **merge** all secondary (**already**) sorted linked lists into one single (unique) sorted linked list and return its pointer (of type `listB_t`).

Define the type `listA_t` and `listB_t` to implement the **linked list** with all necessary structure fields.

Write the entire function using standard C libraries but implement all required personal libraries. Modularize the function adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward code, complex or impossible to understand, will be penalized in terms of the final evaluation.

Exercise 13 (9.0 points)

Write a C function that given a set of tasks, each one with a time duration and a priority, selects the subset of those tasks whose total duration time is less or equal to a given threshold while maximizing the sum of the priorities. The prototype of the function is the following:

```
void opt_task (int n, int *time, int *priority, int th, int *best, int *l, int *max);
```

In the prototype, the following are input parameters: Variable `n` is the number of tasks, `time` the array expresses their duration (in hours), and `priority` the array indicates their priorities (or their rewards, once solved). In the prototype, the output parameters are the following: The array `best` specifies the selected tasks (their indices in the `time` and `priority` arrays), the number of tasks assigned and stored into `best`, `l`, and the variable `max` storing the sum of the tasks' priorities (i.e., the total reward obtained from their selection).

For example, let us suppose that $n = 4$, $\text{times} = \{2, 3, 4, 5\}$, $\text{priority} = \{3, 4, 5, 8\}$, and $\text{th} = 7$. The function should select tasks 0 and 3 whose times are 2 and 5, have a sum $2 + 5 = 7 \leq \text{th}$, and a total priority (reward) of $3 + 8 = 11$. Thus, it should return $\text{best} = \{0, 3\}$, $l = 2$, and $\text{max} = 11$.

Function `opt_task` can be considered a wrapper defining other objects (such as the current solution array and size) and calling the actual recursive function (e.g., `opt_task_r`).

Write the entire function using standard C libraries but implement all required personal libraries. Modularize the function adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward code, complex or impossible to understand, will be penalized in terms of the final evaluation.